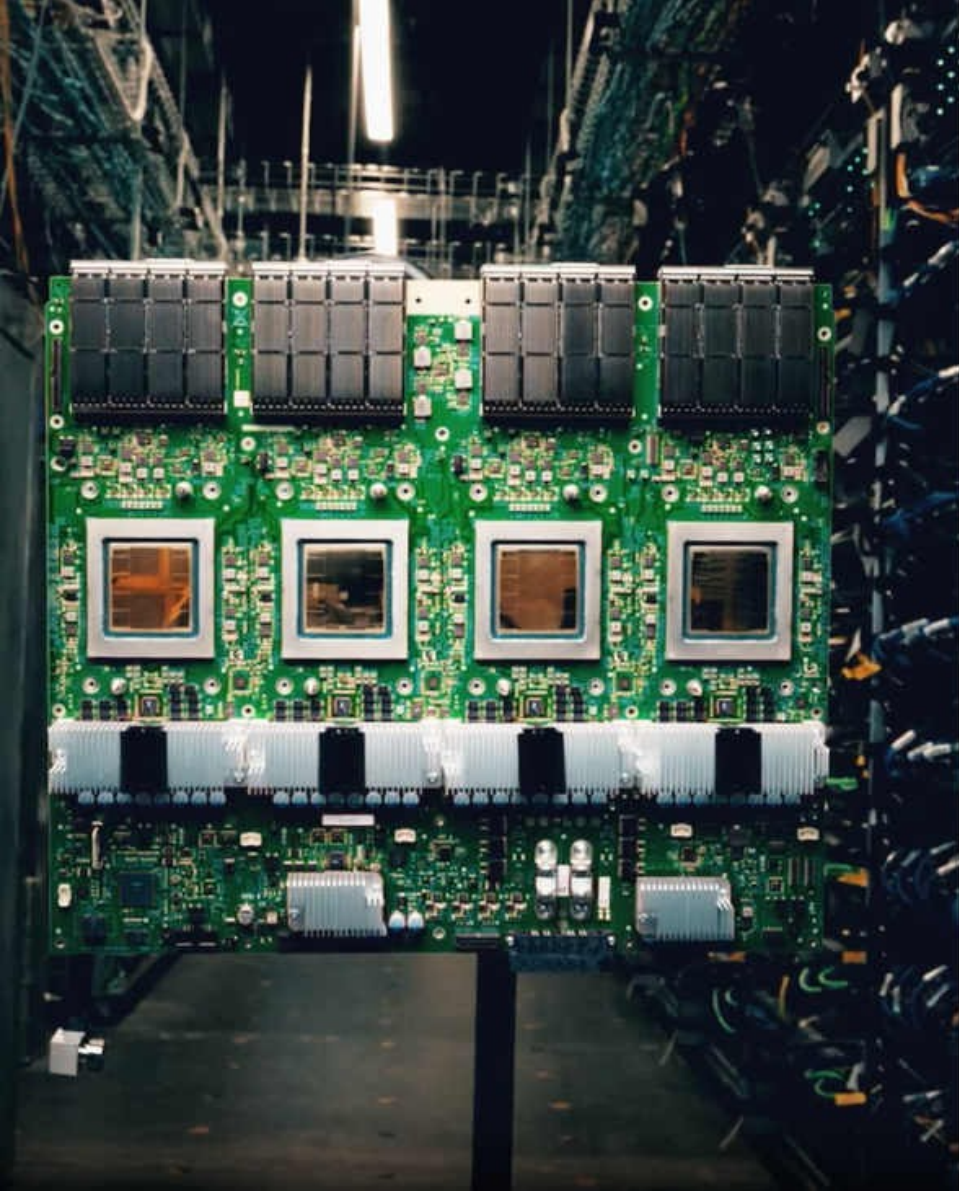


Множење матрица – Data Flow парадигма, GoogleTPU технологија



Електротехнички факултет,
Универзитет у Београду



Мотивација: зашто TPU?

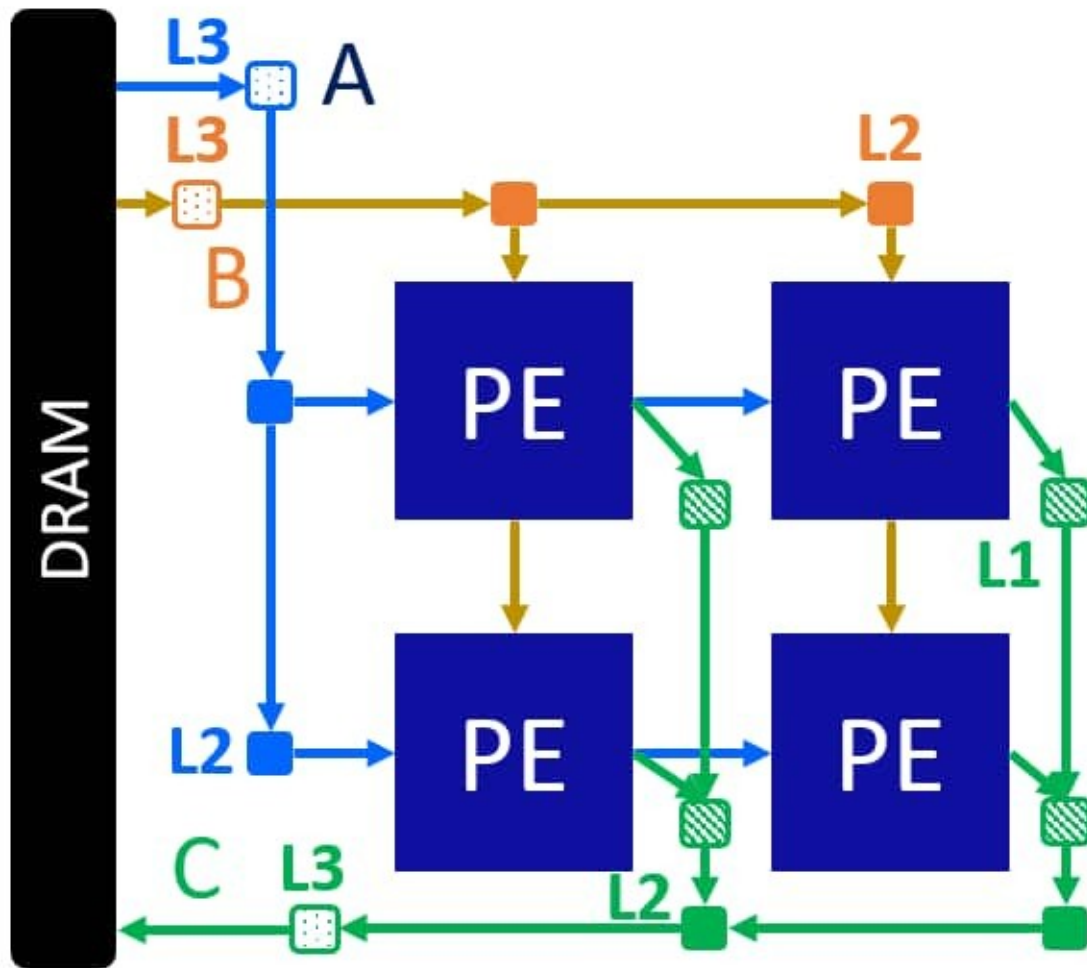
- CPU – дизајниран за опште операције
 - Multiply-accumulate му није јача страна
- GPU – убрзава паралелизам
 - Оптимизован за графику
- TPU – tensor processing unit
 - Специјализован ASIC дизајн
 - ASIC – Application Specific Integrated Circuit

Мотивација: зашто TPU?

- Множење матрица - срж неуралних мрежа
 - 70-90% рачунског времена тренинга
- TPU v4:
 - Red Oak систем
 - 4096 чипова
 - 1.1 exaFLOP/s

TRU архитектура(висок ниво)

- Централна јединица:
 - MXU (Matrix Multiply Unit)
 - 128x129 systolic array
 - 4096 PE (processing element)
 - Сваки PE:
 - Прима податке од суседа
 - Множи
 - Прослеђује даље

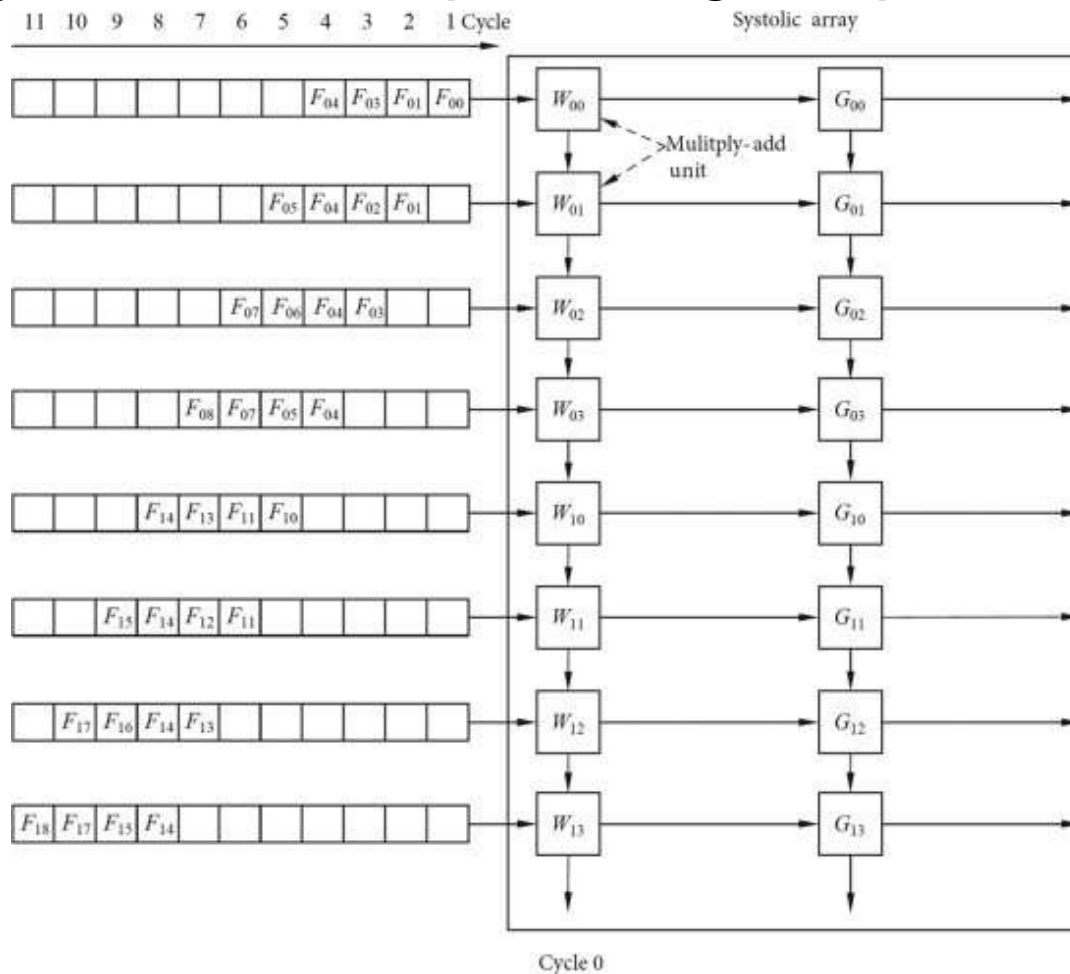


- 2x2 grid PE
- Матрице A, B, C
- Хијерархија мем:
 - L1 кеш
 - L2 кеш
 - L3 кеш

Systolic Array: како ради?

- У мрежу PE чворова:
 - Улаз матрице A утискује се слева
 - Улаз матрице B утискује се одозго
- Сваки PE:
 - Прима парцијални збир одозго
 - Прима елемент матрице слева
 - Множи
 - Акумулира
 - Прослеђује доле

Пропагација података кроз 2D grid кроз 9 циклуса



Systolic Array: како ради?

- У мрежу PE чворова:
 - Улаз матрице A утискује се слева
 - Улаз матрице B утискује се одозго
- Сваки PE:
 - Прима парцијални збир одозго
 - Прима елемент матрице слева
 - Множи
 - Акумулира
 - Прослеђује доле

Google Colab + TPU: Setup

- Colab runtime:
 - Runtime → Change runtime type → TPU
- Доступни су TPU v2-8 (8 језгара)
- TPU има 8 чипова → 8 XLA device-a
 - TPU0..7
- Приступ Pytorch-у:
 - torch_xla библиотека – PyTorch XLA backend за TPU
- Приступ у JAX-у:
 - Нативна подршка
 - TPU је подразумевани акцелератор

JAX: Зашто TPU?

- JAX – NumPy compatible библиотека
- `Jax.jit()` компајлира Python функцију у XLA(Accelerated Linear Algebra)
- Оптимизује меморијски распоред аутоматски
- `Jax.numpy` је drop-in замена за NumPy – исти API
- `mpar()` - паралелизација по свим чиповима TPU-а аутоматски
- `vmap()` - векторизација функција апо димензијама без писања петљи

Код: основни matmul

- Инсталација: `!pip install jax[tpu]` са Google Cloud TPU URL-ом
- Иницијализација: провера уређаја `jax.devices()`
- Креирање матрица: `jax.numpy.ones((N, N))` на TPU device-у
- Егзекуција: позив функције → првим позивом компајлирање
 - Свим наредним брзо извршавање
- Мерење времена `time.time()` уз `jnp.block_until_ready()` за прецизност

Код: Pmap паралелизација

- Вишечипна паралелизација
- `jax.pmap()` аутоматски дистрибуира рачунање на свих 8 TPU чипова
- Матрице се деле по batch димензији: сваки чип добија 1/8 посла
- Резултат: 8x убрзање за batch множење без ручног управљања
- `jax.pmean()` - редукција резултата са свих чипова
- Идеалан за тренинг неуралних мрежа:
 - Сваки чип тренира на делу batch-а

Benchmark: CPU vs GPU vs TPU

- Тестирана величина матрице:
 - 1024x1024, float32 → bf16
 - CPU (Intel Xeon, Colab) ~ 350ms по множењу
 - GPU (T4, Colab) ~ 3.5ms по множењу
 - TPU v2-8 (један чип) ~ 0.8ms по множењу
 - TPU v2-8 (pmap, 8 чипова) ~ 0.12ms за batch од 8 матрица
- Напомена: JAX JIT компајлирање (warm-up) траје 5-15s при првом позиву
 - Мерити тек након тога

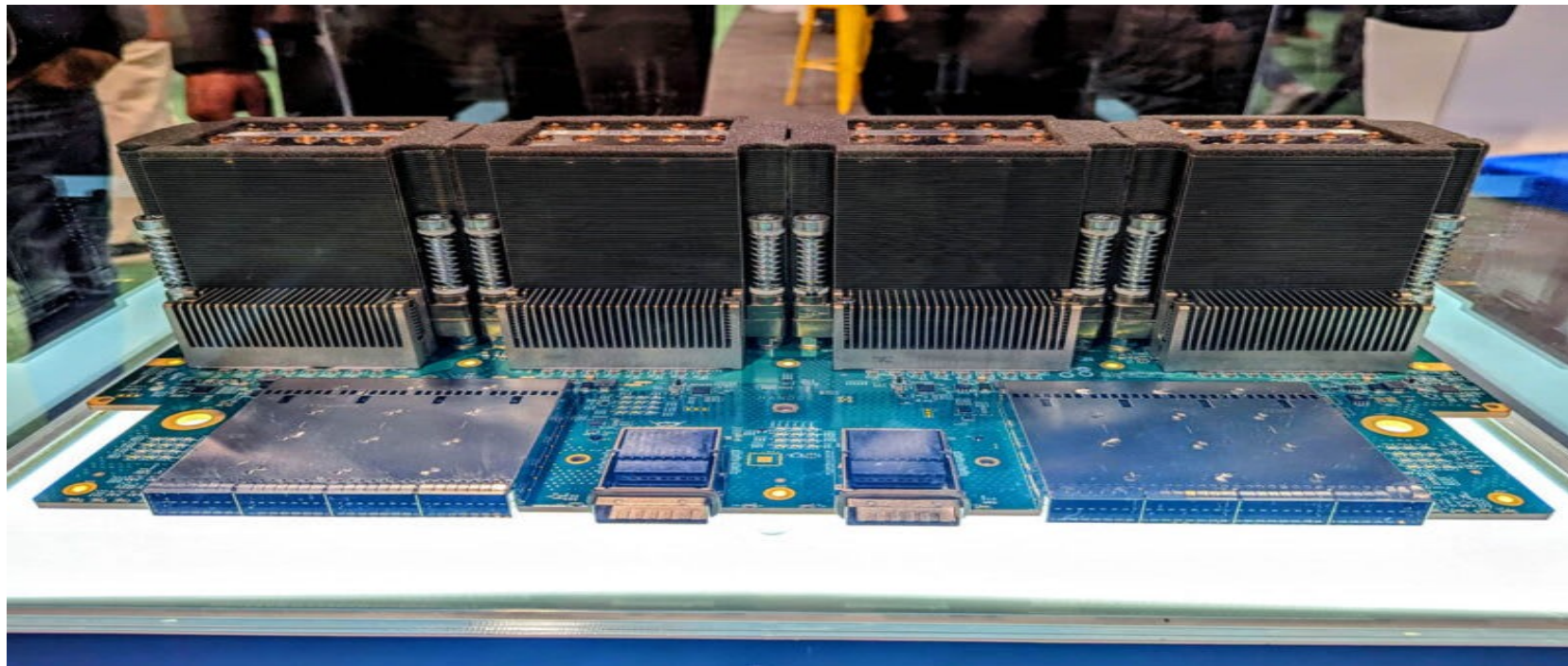
Ограничења и напомене

- TPU преферира bf16 и int8 - float32 операције су спорије
- За пуну MXU искоришћеност
 - Пуњење димензија (padding) на вишекратнике 128 је критично
- Комуникација host ↔ TPU је скупа:
 - Минимизовати трансфер података
 - `block_until_ready()` је обавезан за тачно мерење - JAX је асинхрон
- TPU није увек доступан на Colab-у

Закључак

- TPU systolic array је оптимизован ASIC за тензорске операције
- JAX + jit + pmap = минималан код, максимална искоришћеност TPU
- Кључна лекција:
 - Хардвер архитектура директно диктира стратегију кодирања
- Матрица → MXU → резултат за 1 такт при пуном 128x128 tile-y
- Следећи корак: XLA profiler, custom kernels, JAX shard_map
- Додатна литература:
 - <https://docs.jax.dev/en/latest/pallas/tpu/matmul.html>
 - https://sqtian.com/post/pallas_tpu_matmul_kernel/

"The best way to accelerate a computation is to not do it at all, and the second best way is to do it in hardware." - Jeff Dean, Google



Хвала на пажњи!

Припремили:

Павле Шаренац 2025/3068
sp253068m@student.etf.bg.ac.rs

Никола Николић 2025/3109
nn253109m@student.etf.bg.ac.rs